Nombre A: Achraf Aitbaych **DNI A**:_____

Nombre B: Ismael Mateo Diaz **DNI B**:_____

Asignatura: **Fecha**:

Instrucciones:

Se valorará el orden y limpieza. Una respuesta incomprensible se considerará no respondida. Es obligatorio respetar el formato de entrega. **Todos** los recursos **deben** ser entregados. El incumplimiento supondrá una calificación de cero.

Procura ser breve y responder a todo lo que se pide.
Explicar (brevemente) significa 2-3 líneas. Indicar (brevemente) significa 1-2 líneas. Nombrar significa 1 palabra. La respuesta **solo debe** añadirse después de cada pregunta; en otro lugar supondrá una calificación de cero.

Cualquier tipo de fraude o copia supondrá una calificación de cero.
Lee antes de empezar y analiza los recursos proporcionados. Organiza el tiempo.

Formato de entrega:

Una única carpeta, que será comprimida y entregada en un archivo .zip.
Este documento resuelto por el alumno, en la raíz de dicha carpeta, en .pdf y .docx.

Si es necesario, dentro de la carpeta principal habrá una o varias subcarpetas que incluirán **TODOS los recursos adicionales pedidos, desarrollados y usados**. Los recursos no entregados supondrán una calificación de cero en el ejercicio donde se pidan.

Las capturas deben incluir el nombre del autor y la palabra P3 (ej: Juan Perez P3).
Los recursos se deben organizar en subcarpetas por ejercicios.

Formato mínimo:
dawm2d_opia_p3.zip
|.....dawm2d_opia_p3.pdf (tarea, en pdf)
|.....dawm2d_opia_p3.docx (tarea, en docx)
|.....recursos/ (carpeta para recursos adicionales, pedidos, desarrollados y usados)
|.....ejercicio_n/ (subcarpeta para recursos del ejercicio n)

Instrucciones.
La práctica se realizará en parejas y deberá incluir el nombre de ambos integrantes.

Evaluación.
Todos los apartados (CE) se ponderan por igual en la nota del RA.

PRÁCTICA 3 - Conexión entre la aplicación y el servicio de IA [RA3] v0

Usa la plantilla de código adjunta para completar esta práctica.

- **app.py:** server FastAPI básico con predicción. Igual al de práctica 2.
- **streamlit_app.py:** web Streamlit con una interfaz básica. Llama a app.py.

Puedes incluir una sola versión de **streamlit_app.py** con todos los cambios acumulados en el ejercicio final.

a) Conexión app-servicio de IA

Configura la conexión entre la aplicación Streamlit y tu servicio de IA: indica la URL pública de tu API en Render y envía los datos en formato JSON mediante una petición POST al endpoint `/predict`. Despliega app.py y streamlit_app.py.

Entregable (memoria): una captura de las aplicación Streamlit (streamlit_app.py) y FastAPI (app.py) en Render. El log de las app. Las url del servicio FastAPI y Streamlit en Render.

1. URL del Streamlit en Render: https://streamlit-california-housing-nib6.onrender.com
2. URL de la API en Render: https://proyecto-california-housing-price.onrender.com

Precio de casas en California

0.1.0 OAS 3.1

/openapi.json

default

GET / Health

Health-check: confirma que el servicio está funcionando.

Parameters Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls `Accept` header. Example Value | Schema `"string"`	*No links*

POST /predict Predict

PROJECT

Proyecto-california-housing-price

Production

All (2) | Services (2) | Env Groups (0)

Search resources in Production

SERVICE NAME 2	STATUS	RUNTIME
streamlit-california-housing	Deployed	Python 3
Proyecto-california-housing-price	Deployed	Python 3

+ New



Logs en API

```
11:00:40 AM  [gg52x]  INFO:     188.244.92.243:0 - "GET /favicon.ico HTTP/1.1" 404 Not Found
11:00:41 AM  [gg52x]  INFO:     188.244.92.243:0 - "GET / HTTP/1.1" 200 OK
11:00:55 AM  [gg52x]  INFO:     74.220.48.71:0 - "POST /predict HTTP/1.1" 200 OK
```

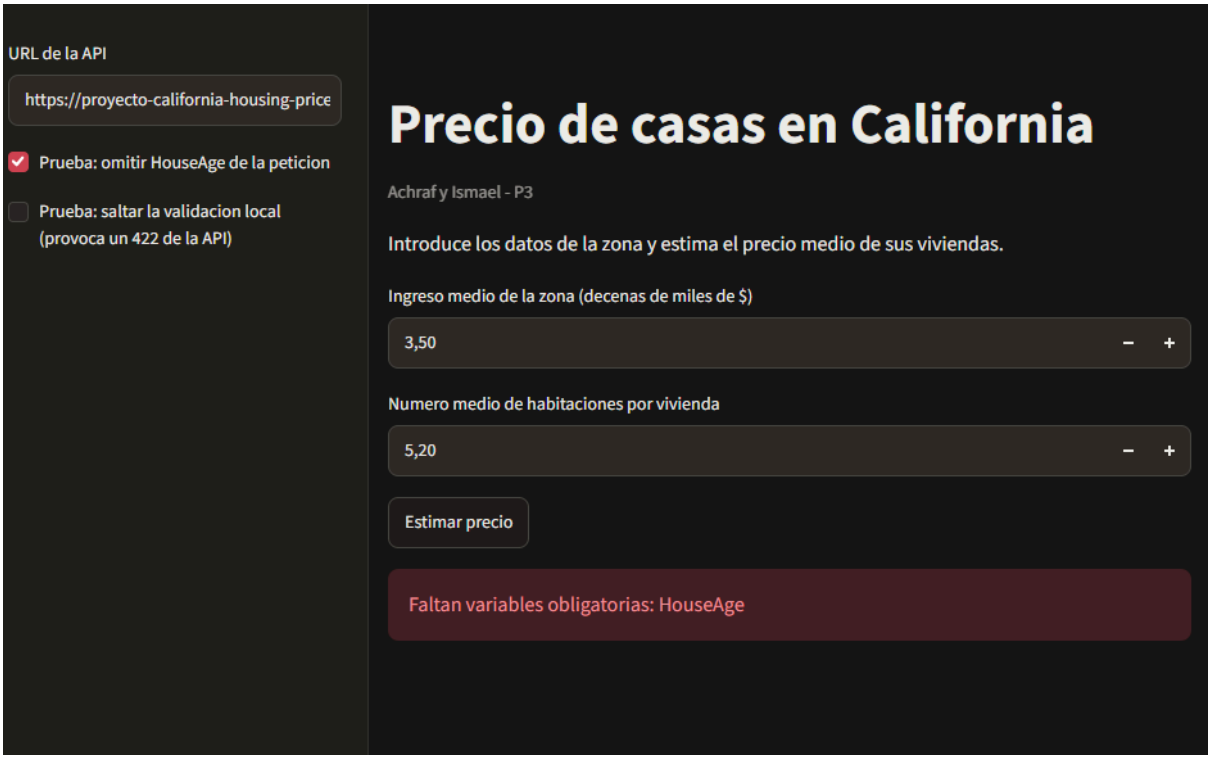
Logs en Streamlit

```
11:51:29 AM  [mxkxf]  INFO:     188.244.92.243:0 - "GET / HTTP/1.1" 200 OK
11:51:34 AM  [mxkxf]  INFO:     74.220.48.71:0 - "POST /predict HTTP/1.1" 200 OK
11:52:08 AM  [mxkxf]  INFO:     74.220.48.71:0 - "POST /predict HTTP/1.1" 200 OK
11:52:19 AM  [mxkxf]  INFO:     74.220.48.71:0 - "POST /predict HTTP/1.1" 200 OK
```

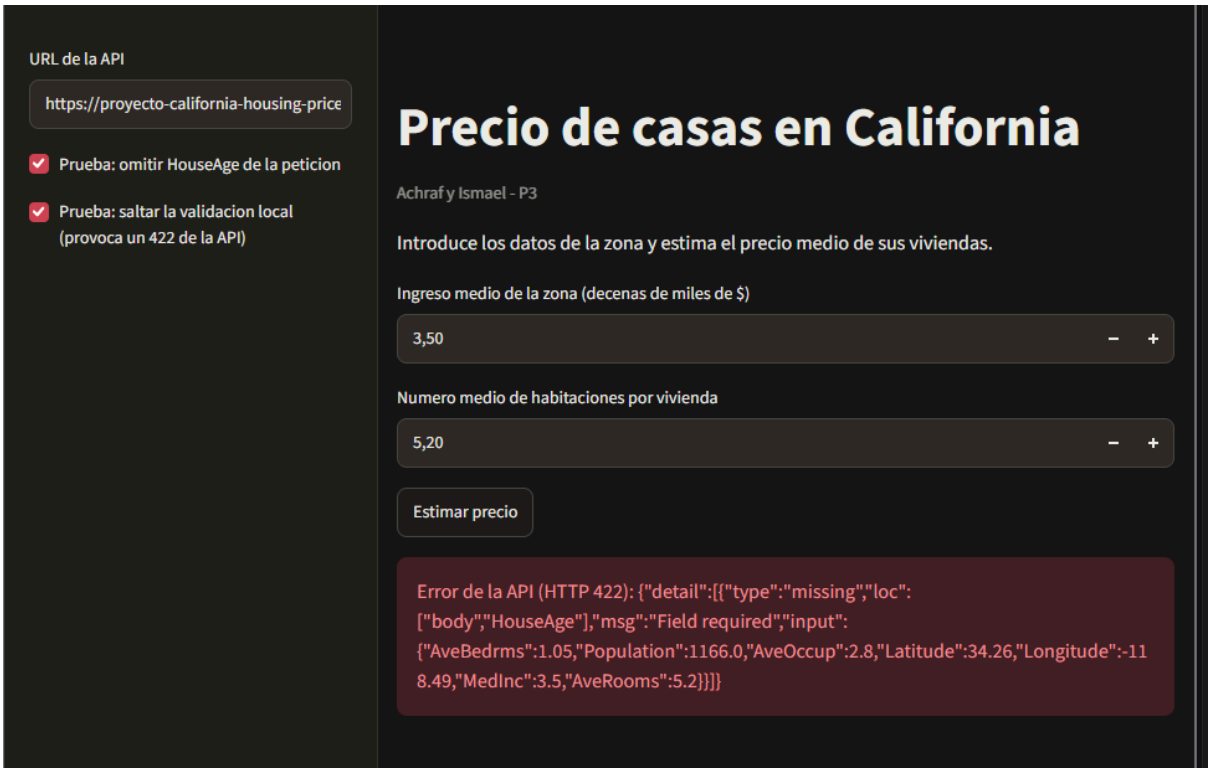
b) Preparación de los datos de entrada
La aplicación solo pide al usuario algunas variables, así que define valores por defecto razonables para el resto. Antes de enviar la petición, valida que contiene las 8 variables que necesita el modelo (MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude); si falta alguna, la aplicación no envía la petición y muestra qué falta. Completa el código faltante.

Entregable (ZIP, memoria): el código. Dos pruebas: una petición válida y otra con un campo faltante, mostrando el resultado de cada una.

1. La aplicación identificó la ausencia de “HouseAge” en el cliente mediante la función “find_missing()”, evitando realizar la llamada a la red.



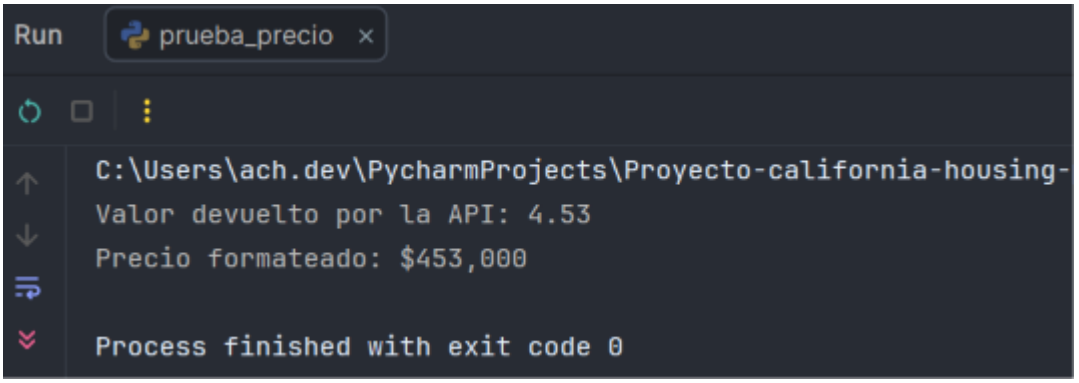
2. Al saltar la validación del cliente, el JSON incompleto llegó al backend, donde el modelo de Pydantic (“HouseFeatures”) rechazó la estructura por falta de “HouseAge” devolviendo el estado “422”.



c) Transformación de los datos de salida

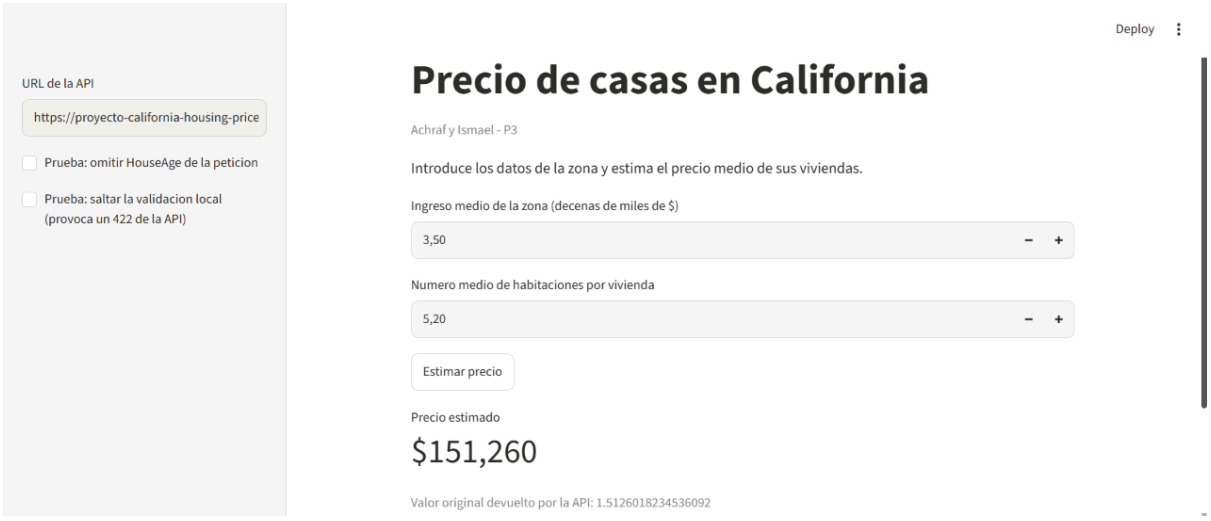
Transforma la respuesta de la API a un formato legible (precio en dólares) antes de mostrarla. Recuerda que la variable objetivo del dataset está en cientos de miles de dólares. Completa el código faltante.

Entregable (ZIP, memoria): el código. Un ejemplo que muestre el valor devuelto por la API y el precio ya formateado que ve el usuario.



d) Diseño de interfaz de usuario
Completa la interfaz de Streamlit para que permita introducir las variables clave (ingreso medio, número de habitaciones).

Entregable (ZIP, memoria): el código. Una captura de la interfaz funcionando con un caso de prueba real.



e) Documentación y control de errores
Añade control de errores a la aplicación (respuestas de error de la API) con mensajes claros para el usuario.
Documenta la API en un README con un ejemplo de petición y de respuesta.

`POST /predict`

Este endpoint recibe mediante JSON las ocho variables necesarias para realizar la predicción del precio medio de las viviendas. Devuelve el resultado en el campo `MedHouseVal`, expresado en cientos de miles de dólares.

Ejemplo de petición

```
{
  "MedInc": 3.5,
  "HouseAge": 29.0,
  "AveRooms": 5.2,
  "AveBedrms": 1.05,
  "Population": 1166.0,
  "AveOccup": 2.8,
  "Latitude": 34.26,
  "Longitude": -118.49
}
```

Ejemplo de respuesta correcta (200)

```
{
"MedHouseVal": 1.5126018234536092
}
```

Entregable (ZIP, memoria): el código completo usado (con todos los cambios completos de `streamlit_app.py` de todos los ejercicios) en el ZIP. El README con la documentación de la API incluida (ZIP). Una prueba que provoque un error controlado, por ejemplo cambiando la URL de la API (memoria).

